Labo 16 — Resources en exploitatie

Theorie — wat een gedeelde cluster van elke applicatie eist: zeggen wat ze verbruikt, een budget aanvaarden, observeerbaar en debugbaar zijn, alleen de rechten hebben die ze nodig heeft, en alleen praten met wie ze moet.

Doelstellingen

- **Requests** en **limits** declareren, de **QoS**-klassen begrijpen, en `OOMKilled` en `Pending` herkennen.
- Afschermen met **namespaces**, en begrenzen met **LimitRange** en **ResourceQuota**.
- Observeren en debuggen: `kubectl top`, `events`, `logs --previous`, `describe`, `kubectl debug`.
- Een minimale **RBAC** lezen en schrijven: ServiceAccount, Role, RoleBinding, `kubectl auth can-i`.
- Het netwerk beperken met een **NetworkPolicy**.

---

1. Een cluster wordt gedeeld

Tot nu toe had je cluster één huurder: jij. In een bedrijf draaien tientallen applicaties en teams op dezelfde nodes. Drie vragen worden levensbelangrijk: hoeveel verbruikt elke Pod (om hem te plaatsen en te beschermen), wie mag wat doen (om niet het Deployment van de buur te verwijderen), en wie mag met wie praten (zodat een lek in de frontend geen toegang geeft tot de database van de boekhouding). Labo 10 stelde die vragen voor één container; de cluster beantwoordt ze met objecten.

> **→ DOCKER-HERHALING**
> `--memory 256m --cpus 0.5` worden `resources.limits`; de OOM killer en code `137` zijn dezelfde (Pods draaien in dezelfde cgroups, labo 10); `podman stats` wordt `kubectl top`; `podman logs --previous` … bestond niet, maar `podman logs` van een container die door `--restart` herstart was, gaf hetzelfde probleem; en `--cap-drop` / `--read-only` komen ongewijzigd terug in de `securityContext` van de Pod.

2. Requests en limits: zeggen wat je verbruikt

```
resources:
  requests: { cpu: "100m", memory: "64Mi" }     # wat de scheduler reserveert
  limits:   { cpu: "500m", memory: "256Mi" }    # wat de cgroup begrenst
```

De **request** is een **reservering**: de scheduler plaatst de Pod alleen op een node waar de som van de requests onder de capaciteit blijft, en zij wordt meegeteld in `Allocated resources` van `kubectl describe node`. De **limit** is het **cgroup-plafond** van labo 10: geheugen erboven doodt de container (`OOMKilled`, code `137`, en de kubelet herstart met `CrashLoopBackOff`), CPU erboven wordt **vertraagd** (*throttling*), nooit gedood. `100m` = een tiende van een core; `64Mi` = 64 mebibyte.

QoS-klasse	Voorwaarde	Lot bij geheugendruk op de node
`Guaranteed`	requests = limits voor CPU en geheugen	als laatste verwijderd
`Burstable`	requests aanwezig, limits anders of gedeeltelijk	na de BestEffort verwijderd
`BestEffort`	geen request en geen limit	als eerste verwijderd

De kubelet berekent de klasse uit de spec (`status.qosClass`); zij bepaalt wie geofferd wordt als een node geheugen tekortkomt. Een database in `BestEffort` is een database die als eerste sneuvelt.

> **→ JAVA**
> De JVM ziet de **limit** als totaal geheugen (labo 10): met `limits.memory: 256Mi` en `-XX:MaxRAMPercentage=75` is de heap 192 Mi, en de rest (metaspace, threads) moet in de overblijvende 64 Mi passen. Een `OOMKilled` op een Java-API komt bijna altijd van een te ambitieus percentage of een te krappe limit. Voor CPU volstaat een request van `100m` om te draaien, maar een opstartende JVM verbruikt tien seconden lang een volledige core: een te lage CPU-`limit` verlengt de opstart en laat de probes falen — veel teams zetten **geen** CPU-limit.

Een te grote request heeft een ander effect: de Pod blijft `Pending` met `FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`. Er gebeurt niets zolang geen enkele node hem kan ontvangen — het vaakst voorkomende symptoom na opschalen.

3. Namespaces, LimitRange, ResourceQuota

De **namespace** scheidt namen (twee `api`'s in twee namespaces), draagt de RBAC-rechten en de quota's, en wordt in één keer verwijderd. De gebruikelijke indeling: een namespace per applicatie en per omgeving (`facturatie-prod`, `facturatie-test`), of een per team. `kube-system` bevat de componenten van de cluster; `default` zou niets mogen bevatten.

Twee objecten begrenzen een namespace. De **LimitRange** legt **standaardwaarden** vast (een Pod zonder `resources` krijgt `defaultRequest` en `default`) en **plafonds** per container (`max`): een Pod die meer vraagt wordt bij de `apply` **geweigerd** (`maximum memory usage per Container is 1Gi, but limit is 4Gi`). De **ResourceQuota** legt een **totaalbudget** vast: som van de requests, van de limits, aantal Pods, Services, PVC's — `kubectl describe quota` toont `Used` en `Hard`. Met een actieve quota **moet** elke Pod resources declareren (of een LimitRange hebben die ze levert).

> **K3S**
> Eén enkele node maakt dit allemaal in één oogopslag zichtbaar: `kubectl describe node` toont de capaciteit van je WSL (wat `.wslconfig` hem gaf) en de requests van alle Pods, ook die van `kube-system`. Op een beheerde cluster zet het platformteam een LimitRange en een ResourceQuota in elke namespace die het je toevertrouwt: een Pod zonder `resources` wordt er geweigerd of krijgt standaardwaarden — daarom declareert een bedrijfsmanifest ze altijd.

4. Observeren en debuggen

Behoefte	Commando
werkelijk verbruik (CPU, geheugen)	`kubectl top nodes`, `kubectl top pods` (metrics-server, meegeleverd door k3s)
wat er gebeurd is	`kubectl get events --sort-by=.lastTimestamp`, `kubectl describe pod` (sectie Events)
waarom de vorige container stierf	`kubectl logs pod --previous`
een container zonder shell verkennen	`kubectl debug pod -it --image=alpine --target=container --profile=general`
een bestand kopiëren	`kubectl cp pod:/pad ./bestand`
alles over een object	`kubectl get pod -o yaml` (status, conditions, `lastState.terminated.reason`)

`kubectl debug` verdient een woordje: het voegt een **efemere container** toe aan de Pod, in dezelfde proces-namespace als het doel (`--target`), met de tools die jij wilt (`alpine`, `busybox`, `nicolaka/netshoot`). Dat is het antwoord op de distroless-image van labo 05: geen shell in de image, een shell ernaast zolang de diagnose duurt. De efemere container verdwijnt met de Pod en komt nooit in het template.

Het leesrooster van een incident blijft dat van labo 12: `kubectl get pods` (status, `RESTARTS`), `describe` (Events, `Last State`), `logs --previous`, `top`; en voor het netwerk de keten van labo 13.

5. RBAC: alleen geven wat nodig is

Elke aanroep van de API server wordt **geauthenticeerd** (certificaat, token, OIDC) en daarna **geautoriseerd** door de **RBAC** (*role-based access control*). Vier objecten:

Object	Reikwijdte	Inhoud
Role / **ClusterRole**	een namespace / de hele cluster	regels: `resources` + `verbs` (`get`, `list`, `watch`, `create`, `update`, `patch`, `delete`)
RoleBinding / **ClusterRoleBinding**	een namespace / de hele cluster	koppelt een rol aan subjecten: gebruikers, groepen, **ServiceAccounts**

Een **ServiceAccount** is de identiteit van een applicatie in de cluster; elke Pod heeft er een (`default` als niets gezegd wordt), waarvan het token gemount wordt in `/var/run/secrets/kubernetes.io/serviceaccount/`. Een applicatie die de API niet aanroept heeft geen enkel recht nodig — en `automountServiceAccountToken: false` ontneemt haar zelfs het token. `kubectl auth can-i <werkwoord> <resource> --as=system:serviceaccount:<ns>:<naam>` test een permissie zonder opnieuw in te loggen, en `--list` somt alles op wat een subject mag.

→ BEVEILIGING

De meegeleverde ClusterRoles (`view`, `edit`, `admin`, `cluster-admin`) dekken de meeste menselijke behoeften: `view` om te lezen zonder de Secrets, `edit` om te deployen, `admin` om de RBAC van de namespace te beheren. `cluster-admin` is de root van de cluster: nooit voor een applicatie, zelden voor een mens. De kubeconfig van k3s **is** `cluster-admin` — vandaar labo 11 en zijn `644`. Het **minste privilege** van labo 10 geldt: een Pod die de API niet nodig heeft, heeft een lege ServiceAccount, en een operator die ze wel nodig heeft, heeft een Role die precies zijn resources opsomt.

6. NetworkPolicy: alleen praten met wie nodig is

Standaard **kan elke Pod elke Pod bereiken**, in alle namespaces (labo 13). Een **NetworkPolicy** selecteert Pods (`podSelector`) en declareert wat toegelaten is als inkomend (`ingress`) en/of uitgaand (`egress`) verkeer; zodra een policy een Pod selecteert, wordt **alles wat niet opgesomd is geweigerd**. Het voorbeeld van het labo: alleen Pods met `app=web` mogen `app=api` bereiken op `8080`; een client zonder dat label krijgt `Connection refused`. Policies stapelen zich op (unie van de toelatingen), en de gangbare praktijk begint met een *deny-all*-policy per namespace, gevolgd door expliciete toelatingen.

→ NETWERK

De NetworkPolicy is een object van de API; het is de **CNI** die ze toepast (of niet). Flannel alleen doet dat niet; k3s bevat een policy-controller (kube-router) die filterregels programmeert op elke node, Calico en Cilium doen hetzelfde met meer verfijning (laag 7, FQDN). Op een cluster waarvan de CNI de policies negeert, wordt het object aanvaard en heeft het **geen effect**: controleer met een test, zoals in het labo. De regels zitten op laag 3/4 (IP, poort): de NetworkPolicy vervangt de authenticatie tussen services niet.

7. In het bedrijf

- **Elk manifest declareert** `resources`, met gemeten requests (`kubectl top` onder belasting) en geheugenlimits; `Guaranteed` voor databases en alles wat niet verwijderd mag worden.
- **Een namespace per applicatie en per omgeving**, met LimitRange en ResourceQuota gezet door het platform; ontwikkelaars krijgen er `edit` op, `view` elders.
- **Applicaties hebben een eigen ServiceAccount**, zonder rechten standaard; operators en agents (CI, GitOps) hebben precieze, geauditeerde rollen.
- **Deny-all + expliciete toelatingen** in NetworkPolicy, minstens in productie; de database aanvaardt alleen de API.
- **Observeerbaarheid** is een dienst van het platform (Prometheus, Loki, Grafana) die leest wat je Pods blootstellen: `stdout`, `/actuator/prometheus`, de Events.

Onthouden

- `requests` reserveren en plaatsen, `limits` begrenzen; geheugen overschreden = `OOMKilled` (137), CPU overschreden = vertraagd; onmogelijke request = `Pending`.
- QoS `Guaranteed` > `Burstable` > `BestEffort`: de volgorde waarin de node Pods offert.
- Namespace = scheidingswand; LimitRange = standaardwaarden en plafonds per container; ResourceQuota = budget van de namespace.

- `top`, `events`, `describe`, `logs --previous`, `debug --target`: de diagnosekoffer, zonder iets aan de images toe te voegen.
- RBAC = Role (wat) + RoleBinding (wie); `auth can-i --as=` om te testen; een applicatie heeft een ServiceAccount zonder rechten.
- NetworkPolicy = witte lijst per label, toegepast door de CNI (k3s doet het); zonder policy praat alles met alles.

Woordenlijst

request / limit: reservering voor de scheduler / cgroup-plafond. — **QoS**: `Guaranteed`, `Burstable`, `BestEffort`. — **`OOMKilled`**: container gedood wegens geheugenoverschrijding (137). — **throttling**: CPU-vertraging aan de limit. — **LimitRange**: standaardwaarden en plafonds per container in een namespace. — **ResourceQuota**: totaalbudget van een namespace. — **metrics-server**: bron van `kubectl top`. — **efemere container**: diagnosecontainer toegevoegd door `kubectl debug`. — **RBAC**: Role, ClusterRole, RoleBinding, ClusterRoleBinding. — **ServiceAccount**: identiteit van een Pod bij de API. — **`kubectl auth can-i`**: permissietest. — **NetworkPolicy**: verkeersregels tussen Pods, toegepast door de CNI.